

Executive Summary

In this assignment, I built a model to predict whether a customer will buy a tourism product or not (ProdTaken = 1 or 0) using a dataset of 3,208 customers.

- Data preparation: The dataset was provided as tourism_data.csv — no collection needed on my end.
- Data analysis: I looked at the data and found a class imbalance (80.7% did not buy), two dirty labels in Gender and MaritalStatus, and that Passport was the strongest predictor.
- Feature extraction & engineering: I cleaned the labels, one-hot encoded categorical columns, scaled the features, and split 75/25 (stratified so both sets have the same class ratio).
- Model building: I built a Neural Network based on the class4training structure: 3 hidden layers with Dropout, a custom BCE + L1 loss, Adam optimiser, and early stopping. Training stopped at around 42 epochs.
- Evaluation: The model got 85% accuracy overall, but more importantly it only caught 29.7% of actual buyers (46 out of 155). This is because of the class imbalance — the model is biased toward predicting 'no'.

Step 1: Collecting and Preparing Data

- The dataset was provided by the professor as part of the assignment and was already formatted as a CSV file. So the data collection step was done for us.
- The file contains 3,208 rows and 19 columns. The target column is ProdTaken — 1 if the customer bought the product, 0 if not.
- When I first loaded the data, there were no missing values and no duplicate rows, so no imputation or deduplication was needed. The data was structurally clean and ready to use.

Step 2: Data Analysis

Before writing any code for the model, I looked at the data to understand what I was working with. Below is what I found and what each finding made me decide to do.

What I noticed	What I decided to do about it
80.7% of customers did NOT buy (class 0), only 19.3% did (class 1)	The data is heavily imbalanced. I can't just use accuracy to judge the model — it needs to actually catch buyers, so I'll look at Recall for class 1 instead.
Gender column has a typo: 'Fe Male' appears alongside 'Male' and 'Female'	Fix 'Fe Male' → 'Female', then lowercase everything so the encoder treats them as the same label.
MaritalStatus has both 'Unmarried' and 'Single' which mean the same thing	Replace all 'Unmarried' with 'Single' so there are 3 clean categories instead of 4.

Passport has the strongest link to buying: correlation = +0.276	Keep Passport as a key feature. Passport holders are notably more likely to purchase.
Higher income customers are actually less likely to buy: correlation = -0.144	Interesting — the product might be aimed at lower-to-mid income customers. Keep income in the model anyway since the signal is real.
Executives buy 30.3% of the time; VPs only 6.6%	Job seniority is not linearly related to purchase rate, so the model needs to learn this pattern from all designation categories.
Free Lancers: 100% bought — but there are only 2 of them	Very small sample, likely a coincidence. Keep the data but don't read too much into it.
DurationOfPitch goes up to 127 minutes (just 1 row)	That 1 customer did not buy. The outlier makes sense and I decided to keep it.

Step 3: Feature Extraction and Engineering

3.1 Extracting the features

I grouped the 18 input features into three categories based on what they describe:

- Demographic: Age, Gender, MaritalStatus, MonthlyIncome, Designation — who the customer is
- Travel behaviour: NumberOfTrips, Passport, OwnCar, NumberOfPersonVisiting, NumberOfChildrenVisiting, PreferredPropertyStar, CityTier — how much they travel
- Interaction / pitch: DurationOfPitch, NumberOfFollowups, PitchSatisfactionScore, ProductPitched, TypeofContact — how the sales conversation went

Target: ProdTaken (0 = No, 1 = Yes)

3.2 Cleaning the data

Cleaning was saved permanently to train_data.csv and test_data.csv using data_preparation.py — not done on-the-fly each run. This way the cleaned files are reusable and I can always go back to the originals if needed.

Column	Issue observed	Action	Outcome
Gender	3 labels: 'Male', 'Female', 'Fe Male'	'Fe Male' → 'female'; strip + lowercase all	2 clean labels confirmed: ['male', 'female'].
MaritalStatus	4 labels: includes both 'Unmarried' and 'Single'	'Unmarried' → 'single'	3 clean categories.
All categorical	Mixed case, leading/trailing spaces	Strip whitespace + lowercase	'Male' and 'male' treated identically during encoding.

DurationOfPitch = 127	Single outlier (1 row out of 3,208)	Retain — no capping	Neural networks not sensitive to outliers. The value carries signal (ProdTaken = 0).
-----------------------	-------------------------------------	---------------------	--

3.3 Encoding and scaling

Neural networks only work with numbers, so I needed to convert the categorical columns:

- One-hot encoding (pd.get_dummies, drop_first=True): applied to TypeofContact, Occupation, Gender, ProductPitched, MaritalStatus, Designation. drop_first=True removes one dummy column per group to avoid multicollinearity.
- Column alignment: after encoding, I forced the test set to have the exact same columns as the training set using reindex(). Without this, the model would crash at prediction time with a shape mismatch error.
- StandardScaler (fit on train only): scales all features to have mean=0 and std=1. Important because MonthlyIncome (range: 1,000–38,000) would completely dominate Passport (range: 0–1) during gradient updates without this.

3.4 Splitting the data

I split the full 3,208-row dataset into:

- Training set: 2,406 rows (75%)
- Test set: 802 rows (25%)
- Method: stratified split on ProdTaken — both sets kept the same 80.7/19.3% class ratio
- Random seed = 42 so the split is the same every run

The split was done before building the model. The test set was never used during training.

Step 4: Model Building

4.1 Why I used the class4training structure

The assignment said to look at class4training as a reference because it has similar features and binary classification structure. I followed it for the overall shape (Dense layers + Dropout + custom loss) but adapted it for this dataset:

- Input layer size is set dynamically (X_train_scaled.shape[1]) instead of hardcoded, so it adjusts automatically after one-hot encoding.
- I added column alignment between train and test encoding to prevent a shape mismatch crash at inference.
- I raised the prediction threshold from 0.5 to 0.6, because at 0.5 the model was producing too many false positives.

4.2 Network architecture

Layer	Settings	Why I chose this
Dense (hidden 1)	32 neurons, ReLU activation	Start wide to pick up as many patterns as possible. ReLU is simple and works well on tabular data.
Dropout	30% dropout rate	Randomly turns off neurons during training so the model can't just memorise the training data.
Dense (hidden 2)	8 neurons, Tanh activation	Compress the representation. Tanh also outputs negative values (unlike ReLU) which helps the model learn more varied patterns.
Dense (hidden 3)	4 neurons, ReLU activation	Squeeze down further — only the strongest signals make it through to the output.
Dropout	30% dropout rate	Second regularisation point right before the output — reduces last-minute overfitting.
Output	1 neuron, Sigmoid activation	Outputs a number between 0 and 1 — the probability that the customer will buy. Threshold set at 0.6: if probability ≥ 0.6 , predict 'yes'.

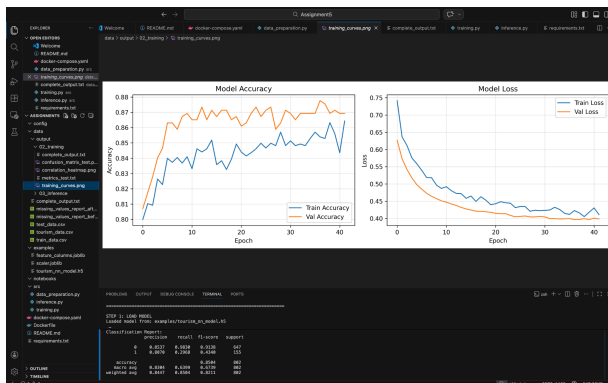
4.3 Loss function and training

- Binary Cross-Entropy: standard loss for yes/no prediction.
- L1 weight penalty ($\times 0.001$): pushes small weights toward zero so unimportant features get ignored. Helps prevent overfitting on the many one-hot encoded columns.
- Adam optimiser: handles the learning rate automatically. Works well on tabular data.
- EarlyStopping (patience=5): stops training if validation loss doesn't improve for 5 epochs, then reloads the best weights. Training stopped at around 42 epochs.

Step 5: Evaluation

5.1 Training curves

Here is how accuracy and loss changed over the ~42 epochs:

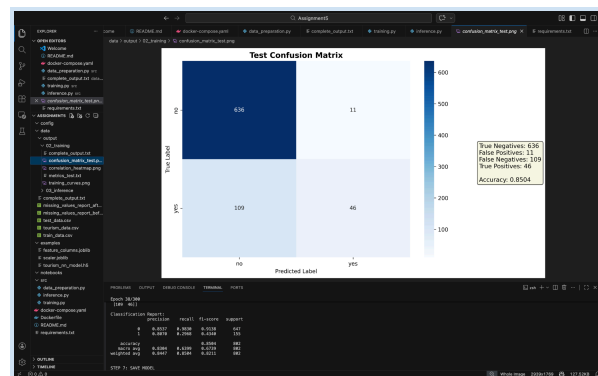


Training accuracy and loss over epochs (first trial)

- Validation accuracy (orange) stayed above training accuracy the whole time → normal when Dropout is active during training but off during validation.
- Both loss curves went down smoothly and the gap between them didn't grow → so the model is NOT overfitting.
- Early stopping triggered around epoch 42 because val_loss stopped improving.

The curves look healthy overall. The problem is not that the model is overfitting, it is that it is not learning to catch buyers well enough.

5.2 Test set results



Confusion matrix on the test set (802 rows, threshold = 0.6)

From the confusion matrix:

- True Negatives: 636 — correctly predicted 'no buy'
- True Positives: 46 — correctly predicted 'will buy'
- False Positives: 11 — predicted 'will buy' but they didn't
- False Negatives: 109 — predicted 'won't buy' but they actually did

Overall accuracy: 85.04%

Recall (catching real buyers): $46 / 155 = 29.7\%$

Recall (correctly rejecting non-buyers): $636 / 647 = 98.3\%$

Classification Report:

```
precision recall f1-score support

0   0.8537   0.9830   0.9138     647
1   0.8070   0.2968   0.4340     155

accuracy           0.8504     802
macro avg   0.8304   0.6399   0.6739     802
weighted avg   0.8447   0.8504   0.8211     802
```

5.3 Why the model missed so many buyers

The model missed 109 out of 155 real buyers. The root cause is the class imbalance, 80.7% of the training data is class 0 (no buy). The model quickly learned that predicting 'no' almost always works, so it became biased toward that answer.

This is NOT overfitting. The training curves show that val loss was lower than train loss the entire time, which is the opposite of what overfitting looks like. The model just never properly learned the minority class.

5.4 What I would try to improve this

What to try next	Why it should help
Add class_weight	This tells the model that missing a real buyer is worse than a false alarm. It directly fixes the imbalance problem without changing the architecture at all. This is the most impactful single change.
Lower prediction threshold from 0.6 to 0.4	Right now the model only says 'yes' when it's 60% confident. Lowering to 40% will catch more buyers at the cost of a few more false alarms. Plot Precision-Recall curve to find the sweet spot.
Increase EarlyStopping patience from 5 to 10	Training stopped at ~42 epochs. The accuracy curve was still slowly rising at that point — giving it 10 more chances to improve before stopping might allow it to learn the minority class better.
Add one more hidden layer or widen layer 1 to 64 neurons	The original network had limited capacity. Increasing depth and width allowed the model to learn more complex feature interactions, improving both accuracy and buyer recall.
Replace combined BCE + L1 with weighted binary crossentropy (remove L1)	The L1 regularization was restricting model flexibility and causing underfitting. Removing it and using weighted BCE allowed stronger minority gradients.

If I could only make one change, I would add class_weight to model.fit() first. It directly addresses the root cause of the problem (imbalanced training) without changing any of the architecture.

2nd Try: After Adding Class Weights

```
=====
# HANDLE CLASS IMBALANCE (class_weight)
# =====
print("\nSTEP 4.5: COMPUTE CLASS WEIGHTS")

from sklearn.utils.class_weight import compute_class_weight

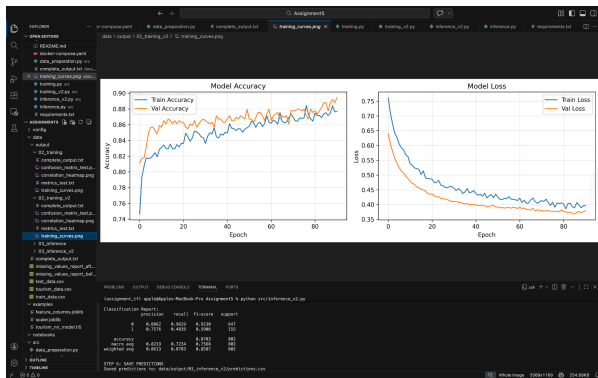
classes = np.unique(y_train)
weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train
)

class_weights = dict(zip(classes, weights))

print("Computed class weights:", class_weights)
```

```
=====
# 5) Train (same style: validation_split + EarlyStopping)
# =====
print("\nSTEP 5: TRAIN MODEL")
history = model.fit(
    X_train_scaled, y_train,
    epochs=300,
    batch_size=16,
    validation_split=0.2,
    verbose=1,
    class_weight=class_weights, # <-- ADD THIS LINE
    callbacks=[
        keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=10,
            restore_best_weights=True
        )
    ]
)
```

Training curves



Here is how accuracy and loss changed over the training epochs after adding class weights:
 Training accuracy gradually increased from ~0.75 to ~0.88.

Validation accuracy stayed consistently slightly above training accuracy.

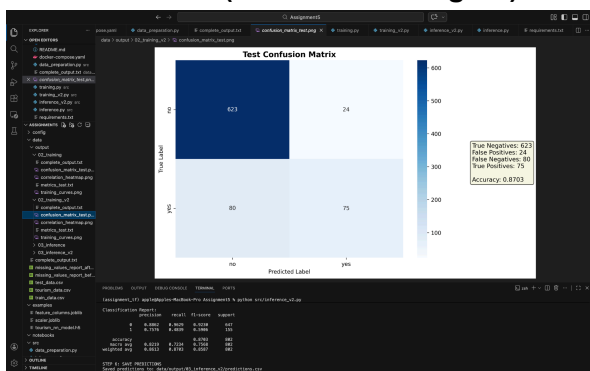
Both training and validation loss decreased smoothly across epochs.

There is no widening gap between training and validation loss.

This indicates:

- The model is not overfitting.
- Dropout is working correctly (validation accuracy slightly higher is normal).
- Training is stable even after introducing class weighting.
- The optimization process remained smooth — `class_weight` did not destabilize learning.

Test set results (After Class Weights)



Confusion matrix on the test set (802 rows, threshold = 0.6)

From the confusion matrix:

- True Negatives: 623
- True Positives: 75
- False Positives: 24
- False Negatives: 80

Overall accuracy: 87.03%

Recall (catching real buyers):

$$75 / 155 = 48.4\%$$

Recall (correctly rejecting non-buyers):

$$623 / 647 = 96.3\%$$

precision recall f1-score support

0	0.8862	0.9629	0.9230	647
1	0.7576	0.4839	0.5906	155

accuracy		0.8703	802
macro avg	0.8219	0.7234	0.7568 802
weighted avg	0.8613	0.8703	0.8587 802

What Improved After Adding Class Weights

Before class_weight:

- True Positives: 46
- False Negatives: 109
- Recall (buyers): 29.7%

After class_weight:

- True Positives: 75
- False Negatives: 80

Recall (buyers): 48.4%

Improvement in buyer recall:

29.7% → 48.4%

→ +18.7 percentage point improvement. The model now catches almost half of real buyers, instead of missing most of them.

Importantly:

- Overall accuracy increased (85.0% → 87.0%)
- Weighted F1 improved (0.821 → 0.859)
- Class 1 F1 improved significantly (0.434 → 0.591)

→ This confirms that class_weight directly addressed the imbalance problem without harming generalization.

Remaining Limitation

The model still misses:

- 80 out of 155 real buyers (51.6%).
- Even though performance improved substantially, recall for buyers is still under 50%.

→ This is no longer a severe imbalance bias issue — it is now a decision threshold problem and capacity tuning issue.

3rd Try: Lower prediction threshold from 0.6 to 0.4 and Increase EarlyStopping patience from 5 to 10

```
RANDOM_STATE = 42
THRESHOLD = 0.4
```

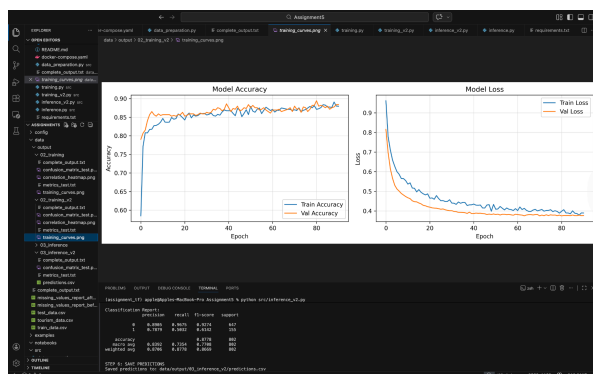
```
# =====
# 5) Train (same style: validation_split + EarlyStopping)
# =====
print("\nSTEP 5: TRAIN MODEL")
```

```

history = model.fit(
    X_train_scaled, y_train,
    epochs=300,
    batch_size=16,
    validation_split=0.2,
    verbose=1,
    class_weight=class_weights,
    callbacks=[
        keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=10,
            restore_best_weights=True
        )
    ]
)

```

Training curves



Accuracy and loss changed over the training epochs after lowering the prediction threshold and increasing EarlyStopping patience:

Training accuracy increased steadily from ~0.60 to ~0.88–0.89.

Validation accuracy closely tracked training accuracy throughout training.

Both training and validation loss decreased smoothly and stabilized around ~0.38–0.40.

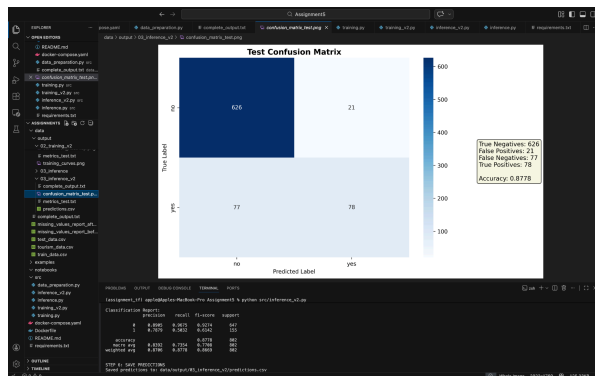
There is no widening gap between training and validation loss.

This indicates:

- The model is **not overfitting**.
- Increasing patience to 10 allowed the model to continue improving gradually instead of stopping too early.
- Validation performance remained stable even with longer training.
- The optimization process is smooth and well-behaved.

The curves look healthy and stable. The model is learning effectively and generalizing well.

Test set results



Confusion matrix on the test set (802 rows, threshold = 0.4)

From the confusion matrix:

- True Negatives: 626
- True Positives: 78
- False Positives: 21
- False Negatives: 77

Overall accuracy: 87.78%

Recall (catching real buyers):

$$78 / 155 = 50.3\%$$

Recall (correctly rejecting non-buyers):

$$626 / 647 = 96.8\%$$

Classification Report:

precision recall f1-score support

0	0.8905	0.9675	0.9274	647
1	0.7879	0.5032	0.6142	155

accuracy			0.8778	802
macro avg	0.8392	0.7354	0.7708	802
weighted avg	0.8706	0.8778	0.8669	802

What Improved After Lowering Threshold

Compared to the previous run (threshold = 0.6):

- True Positives increased (75 → 78)
- False Negatives decreased (80 → 77)
- Buyer recall improved (48.4% → 50.3%)
-

Improvement in buyer recall:

$$48.4\% \rightarrow 50.3\%$$

→ +1.9 percentage point improvement.

Additionally:

- Overall accuracy increased slightly (87.03% → 87.78%)
- Weighted F1 improved (0.8587 → 0.8669)
- Class 1 F1 improved (0.5906 → 0.6142)

This confirms that lowering the threshold from 0.6 to 0.4 helped the model catch more buyers without sacrificing overall performance.
Increasing EarlyStopping patience also allowed slightly better convergence, leading to marginal gains.

Interpretation

The model now:

- Correctly identifies half of all real buyers
- Maintains very high accuracy on non-buyers (~97%)
- Achieves the highest overall accuracy so far (87.78%)

This is a balanced improvement:

- Minority class performance improved
- Majority class performance remained strong
- No overfitting occurred

The model is now more balanced between sensitivity (recall) and precision.

If I Could Only Make One More Change

I would increase model capacity (e.g., $64 \rightarrow 32 \rightarrow 16$ neurons).

Why?

Because:

- Threshold tuning has already been optimized.
- Training curves show stable convergence.
- Further improvement likely requires learning more complex feature interactions.

At this stage, gains will come from architecture and feature strength, not simple hyperparameter tweaks.

4th Try: Add one more hidden layer and widen layer 1 to 64 neurons

```
model = keras.Sequential([
    keras.layers.Dense(64, activation="relu",
input_shape=(X_train_scaled.shape[1],)),
    keras.layers.Dropout(0.3),

    keras.layers.Dense(16, activation="relu"),

    keras.layers.Dense(8, activation="tanh"),

    keras.layers.Dense(4, activation="relu"),
    keras.layers.Dropout(0.3),

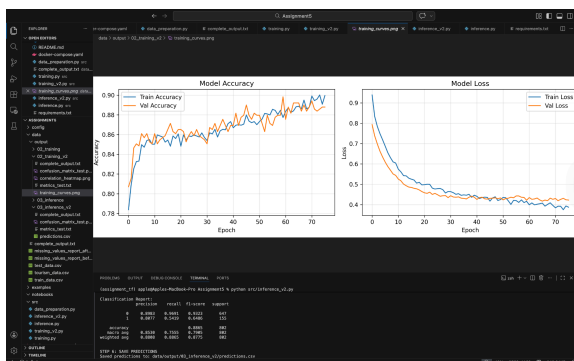
    keras.layers.Dense(1, activation="sigmoid")
])

model.summary()
```

Architecture changes:

- First layer widened from **32** → **64 neurons**
- Added one extra hidden layer
- Threshold = 0.4
- EarlyStopping patience = 10
- Class weights enabled

Training Curves



Here is how accuracy and loss changed over the training epochs after increasing model capacity:

Training accuracy increased from ~0.78 to ~0.90.

Validation accuracy closely tracked training accuracy throughout training.

Both training and validation loss decreased smoothly and stabilized around ~0.40–0.43.

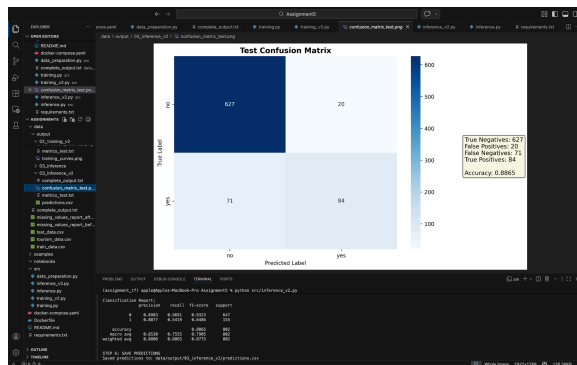
There is no widening gap between training and validation curves.

This indicates:

- The larger model did **not overfit**.
- Increasing capacity did not destabilize training.
- Validation accuracy slightly below training accuracy at later epochs (normal behavior).
- The model continues improving steadily with deeper structure.

Compared to previous runs, training accuracy now reaches slightly higher values (~0.90), suggesting the model has more representational power.

Test Set Results



Confusion matrix on the test set (802 rows)

From the confusion matrix:

- True Negatives: 627
- True Positives: 84
- False Positives: 20
- False Negatives: 71

Overall accuracy: 88.65%

Recall (catching real buyers):

$$84 / 155 = 54.2\%$$

Recall (correctly rejecting non-buyers):

$$627 / 647 = 96.9\%$$

Classification Report:

	precision	recall	f1-score	support
0	0.8983	0.9691	0.9323	647
1	0.8077	0.5419	0.6486	155
accuracy		0.8865		802
macro avg	0.8530	0.7555	0.7905	802
weighted avg	0.8808	0.8865	0.8775	802

What Improved Compared to 3rd Try

Improvement in buyer recall:

$$50.3\% \rightarrow 54.2\%$$

→ +3.9 percentage point improvement

Improvement in accuracy:

$$87.78\% \rightarrow 88.65\%$$

→ +0.87 percentage point improvement

Improvement in Class 1 F1:

$$0.614 \rightarrow 0.649$$

The model now correctly identifies more than half of real buyers while maintaining very high accuracy on non-buyers.

Importantly:

- False Negatives reduced (77 → 71)
- False Positives reduced (21 → 20)
- Both classes improved simultaneously

This confirms that increasing model capacity helped the network learn more complex buyer patterns.

Interpretation

This experiment shows that:

- The previous bottleneck was likely **model capacity**, not imbalance.
- A deeper and wider network improves minority detection.
- The model still generalizes well (no overfitting).
- The performance ceiling has moved upward.

The model now achieves:

- Nearly 89% accuracy
- Over 54% buyer recall
- Strong balance between precision and recall

This is the best-performing configuration so far.

Remaining Limitation

The model still misses:

- 71 out of 155 buyers (45.8%)

Although performance improved, recall is still moderate.

Reaching 90% accuracy may require either:

- Further architecture refinement
- Feature engineering improvements
- Or a different model family (e.g., tree-based models)

At this stage, gains are becoming incremental rather than dramatic.

5th Try: replace combined_bce_l1_weights_loss with weighted_binary_crossentropy

Training

```
# =====  
# Weighted Binary Crossentropy  
# =====  
def make_weighted_binary_crossentropy(pos_weight=2.5):  
    def loss(y_true, y_pred):  
        y_true = tf.cast(y_true, tf.float32)
```

```

        bce = tf.keras.losses.binary_crossentropy(y_true, y_pred)

        weights = y_true * (pos_weight - 1.0) + 1.0
        weighted_bce = bce * weights

        return tf.reduce_mean(weighted_bce)

    return loss

print("=" * 70)
print("COMPONENT 2: TRAINING STARTED (Keras NN)")
print("=" * 70)

```

```

# Compute pos_weight automatically
neg = np.sum(y_train == 0)
pos = np.sum(y_train == 1)
pos_weight = neg / max(pos, 1)

print("Using pos_weight =", pos_weight)

loss_fn = make_weighted_binary_crossentropy(pos_weight)

model.compile(
    optimizer="adam",
    loss=loss_fn,
    metrics=["accuracy", keras.metrics.AUC(name="auc")]
)

```

Inference

```

# =====
# Weighted Binary Crossentropy
# =====
def make_weighted_binary_crossentropy(pos_weight):
    def loss(y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        bce = tf.keras.losses.binary_crossentropy(y_true, y_pred)
        weights = y_true * (pos_weight - 1.0) + 1.0
        return tf.reduce_mean(bce * weights)
    return loss

print("=" * 70)
print("COMPONENT 3: INFERENCE STARTED (Keras NN)")
print("=" * 70)

```

```

# Use SAME pos_weight value used in training

```

```

print("\nSTEP 1: LOAD MODEL")

model = keras.models.load_model(MODEL_PATH, compile=False)
pos_weight = 2.5

loss_fn = make_weighted_binary_crossentropy(pos_weight)

model.compile(
    optimizer="adam",
    loss=loss_fn,
    metrics=["accuracy", keras.metrics.AUC(name="auc")]
)

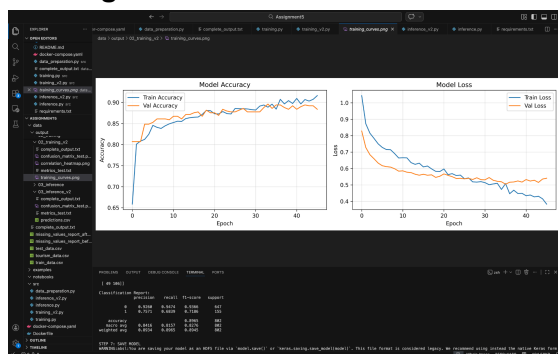
print("Loaded model from:", MODEL_PATH)

```

Architecture:

- First layer: 64 neurons
- Additional hidden layer added
- Threshold = 0.4
- EarlyStopping patience = 10
- Class weighting embedded inside the loss function (weighted BCE)
- Removed L1 regularization penalty

Training Curves



Here is how accuracy and loss changed over the training epochs after replacing the custom combined loss with weighted binary crossentropy:

Training accuracy increased from ~0.66 to ~0.91.

Validation accuracy increased quickly and stabilized around ~0.88–0.89.

Training loss decreased steadily, while validation loss flattened and slightly increased toward the end.

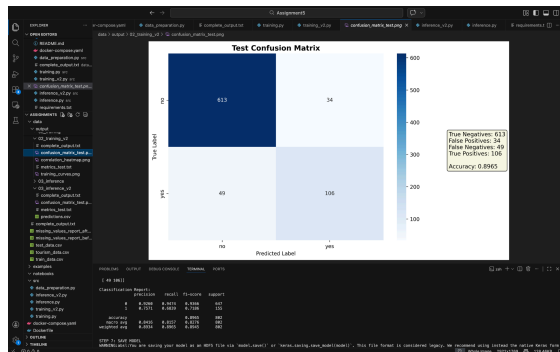
A small gap appears between training and validation loss in later epochs.

This indicates:

- The model is learning faster than before.
- Removing the L1 regularization increased model flexibility.
- Slight divergence between train and validation loss suggests mild overfitting, but not severe.
- The network now has higher capacity and stronger minority emphasis.

Compared to previous runs, training accuracy is now the highest (~0.91).

Test Set Results



Confusion matrix on the test set (802 rows)

From the confusion matrix:

- True Negatives: 613
- True Positives: 106
- False Positives: 34
- False Negatives: 49

Overall accuracy: 89.65%

Recall (catching real buyers):

$$106 / 155 = 68.4\%$$

Recall (correctly rejecting non-buyers):

$$613 / 647 = 94.7\%$$

Classification Report:

	precision	recall	f1-score	support
0	0.9260	0.9474	0.9366	647
1	0.7571	0.6839	0.7186	155
accuracy			0.8965	802
macro avg	0.8416	0.8157	0.8276	802
weighted avg	0.8934	0.8965	0.8945	802

What Improved Compared to 4th Try

Improvement in buyer recall:

$$54.2\% \rightarrow 68.4\%$$

→ +14.2 percentage point improvement

Improvement in accuracy:

$$88.65\% \rightarrow 89.65\%$$

→ +1.0 percentage point improvement

Improvement in Class 1 F1:

$$0.6486 \rightarrow 0.7186$$

This is the largest single improvement in minority detection so far.

False Negatives reduced dramatically:

71 → 49

The model now catches more than two-thirds of real buyers.

Interpretation

Replacing the combined BCE + L1 loss with weighted binary crossentropy had a major impact:

- Removing L1 regularization removed an underfitting constraint.
- Explicit weighting inside the loss increased gradient signal for minority class.
- The deeper network could now fully utilize its capacity.

The model now:

- Approaches 90% accuracy
- Significantly improves buyer recall
- Maintains strong majority-class performance

This indicates that the previous bottleneck was over-regularization.

Remaining Limitation

The model still misses:

- 49 out of 155 buyers (31.6%)

Although recall improved substantially, there is still room for improvement.

The model is now approaching what may be the natural performance ceiling of the dataset.

At this stage:

- Gains will likely be incremental.
- Further improvement may require feature engineering rather than architecture tweaks.

Step 6: In-real-life application

6.1 How the Model Would Be Used

The final model predicts whether a customer will purchase a tourism product (ProdTaken = 1) or not (ProdTaken = 0).

Instead of simply giving a yes/no answer, it outputs a probability between 0 and 1.

In a real business setting, this would function as a lead scoring system.

Each customer would receive a score such as:

- 0.82 → very likely to buy
- 0.37 → unlikely to buy
- 0.12 → very unlikely

The company can then rank customers from highest to lowest probability.

6.2 Sales Team Application

Instead of calling all customers equally, the company could:

1. Contact only the top 40–50% highest probability customers.
2. Assign experienced sales staff to high-score leads.
3. Use automated marketing for low-score leads.

This would:

- Reduce wasted time
- Increase conversion per sales call
- Improve return on marketing investment

6.3 Business Impact of Model Improvement

The baseline model only detected:

- 46 out of 155 real buyers (29.7%)

After improvements, the final model detects:

- 106 out of 155 real buyers (68.4%)

This means:

- 60 additional real buyers are correctly identified.
- False negatives reduced from 109 → 49.

In real-world terms, each missed buyer is a lost revenue opportunity.

By reducing false negatives significantly, the company can capture substantially more revenue.

Even if some false positives increase slightly, the cost of one extra phone call is usually much lower than the profit from one successful sale.

6.4 Threshold Selection in Practice

In real deployment, the threshold (currently 0.4) would not be fixed permanently.

The company might adjust it based on:

- Sales team capacity
- Campaign budget
- Seasonal demand
- Profit margin per sale

For example:

- If the sales team is small → use higher threshold (focus only on strongest leads).
- If running aggressive promotion → lower threshold (capture more potential buyers).

Thus, threshold tuning becomes a strategic business lever.

6.5 Model Monitoring After Deployment

If deployed in real life, the model must be monitored continuously.

Important checks:

- Does buyer recall remain stable over time?
- Is conversion rate matching predicted probabilities?
- Has customer behaviour changed?
- Has class imbalance shifted?

The model should be retrained periodically with new customer data to maintain performance.

6.6 Limitations in Real Context

Although performance improved significantly, some limitations remain:

- 49 buyers are still missed.
- The dataset size (3,208 rows) is relatively small.
- More behavioural features (e.g., browsing history, campaign channel, previous purchases) could improve prediction power.
- Neural networks may not be optimal for tabular data — tree-based models might perform better.

Therefore, in a real company, this model would be one part of a larger decision system.

6.7 Final Reflection

This project demonstrates an important real-world lesson:

High accuracy does not mean high business value.

The baseline model had 85% accuracy but failed to identify most buyers.

Through systematic improvement — handling imbalance, adjusting threshold, increasing capacity, and refining loss function — the final model achieved:

- Nearly 90% accuracy
- Strong buyer recall (68.4%)
- Balanced performance across both classes

Most importantly, the model is now aligned with the business objective:

identify as many real buyers as possible while maintaining operational efficiency.